



My Food Business

Single-Vendor Delivery App — Full Technical Design

DB Schema • REST APIs • Security (BCrypt + Salt) • Game Module • Dev Timeline

Spring Boot • React Native (Expo) • React + Vite • PostgreSQL

Version 2.0 | 2026

1. Project Scope

This is a single-vendor food delivery app — your own food business. There is no restaurant marketplace. You own the kitchen, your staff manages orders from one admin panel, and customers order directly from your branded app.

What this IS	What this is NOT
Your own branded food ordering app	A marketplace like Swiggy / Zomato
One kitchen / one menu	Multi-restaurant platform
You manage all orders from admin	Restaurant owners logging in separately
Delivery agent assigned by you	Third-party delivery fleet
Game module to reward your customers	Complex gamification engine

2. System Architecture

Component	Type	Tech	Who Uses It
Customer App	Mobile (iOS/Android)	React Native (Expo)	Your customers — order food, play games
Admin Panel	Web Dashboard	React + Vite + Shadcn	You — manage orders, menu, coupons, game config
Backend API	REST + WebSocket	Spring Boot + PostgreSQL	All three clients share one backend
Live Tracking	Real-time	WebSocket / Firebase	Customer tracks delivery on map
Payments	Gateway	Razorpay	UPI, cards, COD
Push Alerts	Notifications	Firebase FCM	Order status, game rewards

3. Database Schema

All tables use PostgreSQL. Primary keys are UUID. Timestamps are TIMESTAMPTZ. Soft deletes use `deleted_at`.

3.1 users

Stores all accounts: customers, delivery agents, admin.

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT <code>gen_random_uuid()</code>	Primary key
name	VARCHAR(100)	NOT NULL	Full name
email	VARCHAR(150)	UNIQUE, NULLABLE	Login email (optional if phone login)
phone	VARCHAR(15)	UNIQUE, NOT NULL	Primary login identifier
password_hash	VARCHAR(255)	NULLABLE	BCrypt hash (includes embedded salt)
salt	VARCHAR(100)	NULLABLE	Explicit application-level salt (see §4)
role	ENUM	NOT NULL	customer delivery_agent admin
profile_image_url	TEXT	NULLABLE	Avatar URL
is_active	BOOLEAN	DEFAULT true	Account enabled flag
fcm_token	TEXT	NULLABLE	Firestore push token
otp_code	VARCHAR(10)	NULLABLE	Current OTP (hashed)
otp_expires_at	TIMESTAMPTZ	NULLABLE	OTP expiry time
created_at	TIMESTAMPTZ	DEFAULT NOW()	Account created
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last updated
deleted_at	TIMESTAMPTZ	NULLABLE	Soft delete

3.2 addresses

Column	Type	Constraints	Description
id	UUID	PK	Primary key
user_id	UUID	FK → users.id	Owner
label	VARCHAR(50)	NOT NULL	Home Work Other
address_line1	TEXT	NOT NULL	Street address
address_line2	TEXT	NULLABLE	Apartment / floor

Column	Type	Constraints	Description
city	VARCHAR(100)	NOT NULL	City
pincode	VARCHAR(10)	NOT NULL	Postal code
latitude	DECIMAL(10,7)	NOT NULL	GPS lat
longitude	DECIMAL(10,7)	NOT NULL	GPS lng
is_default	BOOLEAN	DEFAULT false	Default delivery address

3.3 menu_categories

Column	Type	Constraints	Description
id	UUID	PK	
name	VARCHAR(100)	NOT NULL	Starters Mains Desserts
display_order	INTEGER	DEFAULT 0	Sort position
is_active	BOOLEAN	DEFAULT true	Visible to customers
image_url	TEXT	NULLABLE	Category banner image

3.4 menu_items

Column	Type	Constraints	Description
id	UUID	PK	
category_id	UUID	FK → menu_categories.id	Parent category
name	VARCHAR(200)	NOT NULL	Item name
description	TEXT	NULLABLE	Short description
price	DECIMAL(10,2)	NOT NULL	Current price
discounted_price	DECIMAL(10,2)	NULLABLE	Sale price if on offer
image_url	TEXT	NULLABLE	Food photo
is_veg	BOOLEAN	NOT NULL	Veg / Non-veg flag
is_available	BOOLEAN	DEFAULT true	In stock
preparation_time	INTEGER	DEFAULT 15	Minutes to prep
calories	INTEGER	NULLABLE	Calorie info
tags	TEXT[]	NULLABLE	Bestseller Spicy New
display_order	INTEGER	DEFAULT 0	Sort position

3.5 orders

Column	Type	Constraints	Description
id	UUID	PK	
order_number	VARCHAR(20)	UNIQUE NOT NULL	Human ID: ORD-20260001
customer_id	UUID	FK → users.id	Ordering customer
delivery_agent_id	UUID	FK → users.id, NULLABLE	Assigned delivery agent
delivery_address_id	UUID	FK → addresses.id	Where to deliver
status	ENUM	NOT NULL	PLACED→CONFIRMED→PREPARING→READY→PICKED
payment_status	ENUM	NOT NULL	pending paid failed refunded
payment_method	ENUM	NOT NULL	cod upi card wallet
razorpay_order_id	VARCHAR(100)	NULLABLE	Gateway reference
razorpay_payment_id	VARCHAR(100)	NULLABLE	Payment confirmation
subtotal	DECIMAL(10,2)	NOT NULL	Items total
delivery_fee	DECIMAL(10,2)	DEFAULT 0	Delivery charge
discount_amount	DECIMAL(10,2)	DEFAULT 0	Coupon / game discount
tax_amount	DECIMAL(10,2)	DEFAULT 0	GST
total_amount	DECIMAL(10,2)	NOT NULL	Final charged amount
coupon_id	UUID	FK → coupons.id, NULLABLE	Applied coupon
special_instructions	TEXT	NULLABLE	Customer notes
estimated_delivery_at	TIMESTAMPTZ	NULLABLE	ETA
delivered_at	TIMESTAMPTZ	NULLABLE	Actual delivery time
created_at	TIMESTAMPTZ	DEFAULT NOW()	Order placed

3.6 order_items

Column	Type	Constraints	Description
id	UUID	PK	
order_id	UUID	FK → orders.id	Parent order
menu_item_id	UUID	FK → menu_items.id	Item ordered

Column	Type	Constraints	Description
item_name	VARCHAR(200)	NOT NULL	Snapshot of name at order time
unit_price	DECIMAL(10,2)	NOT NULL	Price at time of order
quantity	INTEGER	NOT NULL	Qty
total_price	DECIMAL(10,2)	NOT NULL	qty × unit_price

3.7 delivery_tracking

Stores periodic GPS pings from the delivery agent for live map tracking.

Column	Type	Constraints	Description
id	UUID	PK	
order_id	UUID	FK → orders.id	Related order
agent_id	UUID	FK → users.id	Delivery agent
latitude	DECIMAL(10,7)	NOT NULL	Agent GPS lat
longitude	DECIMAL(10,7)	NOT NULL	Agent GPS lng
recorded_at	TIMESTAMPTZ	DEFAULT NOW()	When ping was recorded

i This table grows fast. Purge records older than 7 days with a scheduled job. Only the latest ping is shown on map.

3.8 reviews

Column	Type	Constraints	Description
id	UUID	PK	
order_id	UUID	FK, UNIQUE	One review per order
customer_id	UUID	FK → users.id	Reviewer
food_rating	SMALLINT	CHECK 1-5	Food quality
delivery_rating	SMALLINT	CHECK 1-5	Delivery experience
comment	TEXT	NULLABLE	Written review
created_at	TIMESTAMPTZ	DEFAULT NOW()	

3.9 coupons

Covers both manually created coupons (admin) and auto-generated game reward coupons.

Column	Type	Constraints	Description
id	UUID	PK	
code	VARCHAR(20)	UNIQUE NOT NULL	e.g. GAME-L2-X7K9P3

Column	Type	Constraints	Description
source	ENUM	NOT NULL	manual game_reward
game_level	SMALLINT	NULLABLE	Game level that generated this (1–5)
discount_type	ENUM	NOT NULL	percentage flat
discount_value	DECIMAL(10,2)	NOT NULL	Amount or percent off
max_discount	DECIMAL(10,2)	NULLABLE	Cap for percentage coupons
min_order_amount	DECIMAL(10,2)	DEFAULT 0	Minimum cart required
assigned_to_user_id	UUID	FK → users.id, NULLABLE	NULL = public coupon
usage_limit	INTEGER	DEFAULT 1	How many times usable
times_used	INTEGER	DEFAULT 0	Redemption counter
valid_from	TIMESTAMPTZ	NOT NULL	Start date
valid_until	TIMESTAMPTZ	NOT NULL	Expiry — game coupons expire in X hours
is_active	BOOLEAN	DEFAULT true	Enable/disable

3.10 game_levels_config

Editable by admin from dashboard — controls discount % per level and coupon expiry duration.

Column	Type	Constraints	Description
id	UUID	PK	
level	SMALLINT	UNIQUE NOT NULL	Level number 1 to 5
level_name	VARCHAR(50)	NOT NULL	Newbie Explorer Regular Foodie Champion
min_games_played	INTEGER	NOT NULL	Min cumulative games to reach this level
discount_type	ENUM	NOT NULL	percentage flat
discount_value	DECIMAL(10,2)	NOT NULL	Reward coupon discount amount — set by admin
max_discount	DECIMAL(10,2)	NULLABLE	Cap for percentage (admin configures)
coupon_expiry_hours	INTEGER	NOT NULL	Hours before unused coupon expires
updated_by	UUID	FK → users.id	Admin who last changed the config
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last config change

Level	Name	Min Games Played	Example Discount (Admin Configurable)	Coupon Expiry
1	Newbie	0	5% off	24 hours
2	Explorer	5	10% off	36 hours
3	Regular	15	15% off	48 hours
4	Foodie	30	₹50 flat off	72 hours
5	Champion	50	₹100 flat off	96 hours

3.11 customer_game_profile

One row per customer — tracks their gaming progress and current level.

Column	Type	Constraints	Description
id	UUID	PK	
user_id	UUID	FK → users.id, UNIQUE	One profile per customer
current_level	SMALLINT	DEFAULT 1	Active game level (1–5)
total_games_played	INTEGER	DEFAULT 0	Cumulative games across all types
last_played_at	TIMESTAMPTZ	NULLABLE	Last game session timestamp
current_coupon_id	UUID	FK → coupons.id, NULLABLE	Latest unspent game coupon

3.12 game_sessions

One record per game play — full audit trail for level progression logic.

Column	Type	Constraints	Description
id	UUID	PK	
user_id	UUID	FK → users.id	Player
game_type	ENUM	NOT NULL	spin_wheel scratch_card trivia_quiz
level_at_play	SMALLINT	NOT NULL	Level when session started
score	INTEGER	NULLABLE	Game score if applicable
won_coupon_id	UUID	FK → coupons.id, NULLABLE	Coupon awarded in this session
played_at	TIMESTAMPTZ	DEFAULT NOW()	Session timestamp

3.13 notifications

Column	Type	Constraints	Description
id	UUID	PK	
user_id	UUID	FK → users.id	Recipient
type	VARCHAR(50)	NOT NULL	order_update game_reward promo system
title	VARCHAR(200)	NOT NULL	Push title
body	TEXT	NOT NULL	Push body
data_json	JSONB	NULLABLE	Extra payload (order_id, coupon_code, etc.)
is_read	BOOLEAN	DEFAULT false	
created_at	TIMESTAMPTZ	DEFAULT NOW()	

4. Security — BCrypt + Salting

4.1 How BCrypt Works

BCrypt is a password hashing function that internally embeds a salt in the final hash string. This means every call to `BCrypt.hashpw()` with the same password produces a different hash — making rainbow table attacks useless.

BCrypt Hash Anatomy	Example Value
Algorithm version	\$2a\$
Cost factor (work factor)	12\$
22-char embedded salt	EixZaYVK1fsbw1ZfbX3OXe
31-char hash body	.suMSoseXTEDbCVmHLSEe2u
Full stored value	\$2a\$12\$EixZaYVK1fsbw1ZfbX3OXe.suMSoseXTEDbCVmHLSEe2u

i

BCrypt with cost factor 12 takes ~250ms per hash — fast enough for login, but too slow for brute-force attacks at scale.

4.2 Why We Add an Application-Level Salt

BCrypt already embeds a salt. Adding an explicit application-level salt in the `users.salt` column provides an extra layer of defence. Even if your database is leaked, attackers cannot crack passwords without also knowing your application-level salt — which lives only in your server environment variables.

Strategy: `FINAL_HASH = BCrypt( password + application_salt + user_specific_salt, cost=12 )`

Salt Layer	Where Stored	Purpose
BCrypt embedded salt	Inside the hash string in DB	Prevents rainbow tables, auto-handled by BCrypt
User-specific salt	<code>users.salt</code> column in DB	Unique per user, prevents identical passwords matching
Application-level secret	Environment variable (not in DB)	Breaks attacks even with full DB dump

4.3 Spring Boot Implementation

```
// application.properties
security.app.pepper=CHANGE_THIS_TO_RANDOM_64_CHAR_SECRET

// PasswordService.java
@Service
public class PasswordService {
```

```
@Value("${security.app.pepper}")
private String pepper;

private final BCryptPasswordEncoder encoder = new
BCryptPasswordEncoder(12);

public String[] hashPassword(String rawPassword) {
    String userSalt = UUID.randomUUID().toString().replace("-", "");
    String combined = rawPassword + userSalt + pepper;
    String hash = encoder.encode(combined);
    return new String[]{ hash, userSalt }; // store both
}

public boolean verify(String rawPassword, String storedHash, String
userSalt) {
    String combined = rawPassword + userSalt + pepper;
    return encoder.matches(combined, storedHash);
}
}
```

⚠ Never log passwords, salts, or pepper values. Store the pepper ONLY in environment variables — not in code or version control.

5. Customer Game Module

5.1 Overview

The Game Module lives inside the Customer App as a dedicated "Games" tab. Playing games rewards customers with discount coupons. The harder the level, the better the coupon. Level is determined purely by how many total games the customer has played — no skill barrier, just engagement reward.

5.2 Three Games

Game	Type	Description	Win Condition
 Spin the Wheel	Luck	Customer taps to spin an animated wheel with food emojis. Wheel slows and lands on a segment.	Every spin wins — segment determines coupon tier
 Scratch Card	Reveal	Customer scratches a virtual card using finger swipe to reveal a hidden discount reward underneath.	Every scratch wins — amount varies by level
 Food Trivia	Skill	Customer answers 3 quick food-themed MCQ questions (e.g. "Which spice makes turmeric yellow?").	Win if 2 out of 3 correct — trivia gives biggest coupon

i All three games are available at every level. The level only determines the coupon reward value — not the game difficulty or accessibility.

5.3 Level Progression Logic

Scenario	Outcome
Customer plays for first time (0 games played)	Assigned Level 1. Gets Level 1 coupon on win.
Customer plays again — total games now 5	System checks game_levels_config. Moves to Level 2.
Customer wins coupon and uses it before expiry	Counts as game played. Next play may level up.
Customer wins coupon but it expires unused	Coupon expires. Customer stays at same level. Next play gives same level coupon.
Customer uses coupon then plays again	total_games_played increments. Level recalculated from config table.
Admin changes level thresholds in dashboard	Next time level is recalculated, new thresholds apply.

Level Calculation Logic (runs after every game session):

```
// GameService.java
public void updatePlayerLevel(UUID userId) {
    CustomerGameProfile profile = gameProfileRepo.findById(userId);
    int totalPlayed = profile.getTotalGamesPlayed();
```

```

    // Fetch all levels sorted descending – pick highest level customer
    qualifies for
    List<GameLevelConfig> levels =
    gameLevelConfigRepo.findAllByOrderByMinGamesPlayedDesc();

    int newLevel = 1;
    for (GameLevelConfig lvl : levels) {
        if (totalPlayed >= lvl.getMinGamesPlayed()) {
            newLevel = lvl.getLevel();
            break;
        }
    }
    profile.setCurrentLevel(newLevel);
    gameProfileRepo.save(profile);
}

```

5.4 Coupon Generation

Every time a customer wins a game, a new coupon is auto-generated in the coupons table with a random unique code.

```

// CouponGeneratorService.java
public Coupon generateGameCoupon(int level, UUID userId) {
    GameLevelConfig config = gameLevelConfigRepo.findByLevel(level);

    String code = "GAME-L" + level + "-" + generateRandomCode(6);
    // e.g.  GAME-L2-X7K9P3

    Coupon coupon = new Coupon();
    coupon.setCode(code);
    coupon.setSource(CouponSource.GAME_REWARD);
    coupon.setGameLevel(level);
    coupon.setDiscountType(config.getDiscountType());
    coupon.setDiscountValue(config.getDiscountValue());
    coupon.setMaxDiscount(config.getMaxDiscount());
    coupon.setAssignedToUserId(userId); // personal coupon
    coupon.setUsageLimit(1);
    coupon.setValidFrom(Instant.now());

    coupon.setValidUntil(Instant.now().plusSeconds(config.getCouponExpiryHours() *
    3600L));
    coupon.setActive(true);

    return couponRepo.save(coupon);
}

private String generateRandomCode(int length) {
    return RandomStringUtils.randomAlphanumeric(length).toUpperCase();
}

```

⚠ If a customer already has an active unexpired game coupon, do NOT generate a new one. Prompt them to use or let the existing one expire first.

5.5 Admin Configuration (Dashboard)

Admin can change discount values per level from the Admin Dashboard without any code deployment:

Admin Can Change	Effect
Discount type (percentage / flat)	All new coupons for that level use the new type
Discount value per level	Immediately applies to next generated coupon for that level
Coupon expiry hours per level	New coupons generated after change use new expiry
Level name (display only)	Shows new name in customer app next refresh
Min games played threshold	Level recalculation uses new threshold immediately

6. REST API Reference

Base URL: <https://api.yourfoodapp.com/api/v1>

All protected routes require: Authorization: Bearer <JWT>

6.1 Auth APIs

Method	Endpoint	Auth	Description
POST	/auth/send-otp	Public	Send OTP to phone number
POST	/auth/verify-otp	Public	Verify OTP → create account or login
POST	/auth/login	Public	Email + password login (admin / agents)
POST	/auth/refresh	Public	Refresh JWT using refresh token
POST	/auth/logout	Bearer	Invalidate refresh token
GET	/auth/me	Bearer	Get own profile

6.2 Customer — Menu & Ordering

Method	Endpoint	Auth	Description
GET	/menu/categories	Bearer	List all active categories
GET	/menu/categories/{id}/items	Bearer	Items in a category
GET	/menu/items/{id}	Bearer	Single item detail
GET	/menu/search?q=	Bearer	Search items by name
POST	/orders	Bearer	Place order (cart in request body)
GET	/orders	Bearer	My order history
GET	/orders/{id}	Bearer	Order detail + current status
PATCH	/orders/{id}/cancel	Bearer	Cancel before PREPARING
POST	/orders/{id}/review	Bearer	Submit review post-delivery
GET	/orders/{id}/tracking	Bearer	Latest agent GPS location
WS	/ws/orders/{id}	Bearer	Real-time status + location stream

6.3 Customer — Addresses & Profile

Method	Endpoint	Auth	Description
GET	/users/profile	Bearer	Get profile
PUT	/users/profile	Bearer	Update name, avatar
GET	/users/addresses	Bearer	List addresses
POST	/users/addresses	Bearer	Add address

Method	Endpoint	Auth	Description
PUT	/users/addresses/{id}	Bearer	Update address
DELETE	/users/addresses/{id}	Bearer	Remove address
PATCH	/users/addresses/{id}/default	Bearer	Set default

6.4 Customer — Payments & Coupons

Method	Endpoint	Auth	Description
POST	/payments/create-order	Bearer	Create Razorpay order → get razorpay_order_id
POST	/payments/verify	Bearer	Verify HMAC signature, confirm payment in DB
POST	/coupons/validate	Bearer	Check if coupon code is valid for current cart

6.5 Game Module APIs

Method	Endpoint	Auth	Description
GET	/game/profile	Bearer	Get level, total games played, active coupon
GET	/game/levels	Bearer	Get all level configs (names, thresholds, rewards)
POST	/game/spin-wheel/play	Bearer	Record spin session → returns coupon if won
POST	/game/scratch-card/play	Bearer	Record scratch session → returns coupon
POST	/game/trivia/question	Bearer	Get 3 random trivia questions
POST	/game/trivia/submit	Bearer	Submit answers → returns coupon if 2/3 correct
GET	/game/history	Bearer	My past game sessions (paginated)
GET	/game/coupon/active	Bearer	Check if I have an active game coupon

6.6 Admin APIs

Method	Endpoint	Auth	Description
GET	/admin/dashboard	Admin	Summary: orders today, revenue, active customers
GET	/admin/analytics/revenue	Admin	Revenue chart — daily/weekly/monthly
GET	/admin/analytics/orders	Admin	Orders volume and status breakdown

Method	Endpoint	Auth	Description
GET	/admin/analytics/games	Admin	Game plays, coupon usage, conversion rate
GET	/admin/orders	Admin	All orders with status + customer filters
GET	/admin/orders/{id}	Admin	Order detail
PATCH	/admin/orders/{id}/confirm	Admin	Confirm order
PATCH	/admin/orders/{id}/assign-agent	Admin	Assign delivery agent to order
GET	/admin/menu/categories	Admin	List categories
POST	/admin/menu/categories	Admin	Create category
PUT	/admin/menu/categories/{id}	Admin	Update category
DELETE	/admin/menu/categories/{id}	Admin	Delete category
POST	/admin/menu/items	Admin	Add menu item
PUT	/admin/menu/items/{id}	Admin	Update item
PATCH	/admin/menu/items/{id}/availability	Admin	Toggle availability
DELETE	/admin/menu/items/{id}	Admin	Delete item
GET	/admin/coupons	Admin	List all coupons (manual + game)
POST	/admin/coupons	Admin	Create manual coupon
DELETE	/admin/coupons/{id}	Admin	Deactivate coupon
GET	/admin/game/levels	Admin	Get all level configs
PUT	/admin/game/levels/{level}	Admin	Update discount/expiry for a level
GET	/admin/users	Admin	List customers + delivery agents
PATCH	/admin/users/{id}/deactivate	Admin	Deactivate user account
GET	/admin/delivery-agents	Admin	List agents with availability status

7. Incremental Development Timeline

This is a solo developer timeline using Spring Boot + React Native (Expo) + React + Cursor AI. AI-assisted coding reduces boilerplate by ~60%.

Without AI (Solo)	With AI + Cursor	With AI + 1 Teammate
5–7 months	10–12 weeks	6–7 weeks

PHASE 1 Foundation + Auth + Core Backend (Weeks 1–2)

Goal: Running Spring Boot project with auth, DB, and deploy pipeline.

Week	Tasks	Deliverable
Week 1	Spring Boot project setup, PostgreSQL schema (users, addresses, menu, orders, coupons tables), BCrypt + salt password service, JWT auth with refresh tokens, OTP send/verify endpoints	Postman-tested auth APIs
Week 2	Deploy to Railway/Render, set up GitHub Actions CI/CD, configure environment variables (pepper, JWT secret, DB URL), test all auth flows end to end, set up S3 bucket for image uploads	Live backend URL + CI pipeline

PHASE 2 Customer App — Core Ordering Flow (Weeks 3–5)

Goal: Customer can browse menu, order food, pay, and see order status.

Week	Tasks	Deliverable
Week 3	Expo React Native setup, navigation structure (Bottom tabs: Home, Orders, Profile), Menu screen with categories + items, Cart screen with add/remove logic, Checkout screen with address selection	Clickable app with menu + cart
Week 4	Razorpay integration (UPI + card), Order placement API connection, Order detail screen with status timeline (Placed → Confirmed → Preparing → Out for delivery → Delivered), Firebase push notifications for status changes	Customer can place and pay for order

Week	Tasks	Deliverable
Week 5	OTP login screen, Profile + saved addresses CRUD, Order history screen, Review/rating screen post-delivery, Basic UI polish with Tailwind/NativeWind	Full customer ordering flow working end to end

PHASE 3 Live Tracking + Delivery Agent Flow (Weeks 6–7)

Goal: Customer sees live delivery map. Agent app to manage deliveries.

Week	Tasks	Deliverable
Week 6	WebSocket setup in Spring Boot for order updates, Delivery agent mobile screens (Expo): incoming assignment, accept/decline, start delivery, GPS ping every 30 seconds, delivery_tracking table inserts, Google Maps integration in customer app	Agent can accept delivery and customer sees live map pin
Week 7	Smooth map animation between GPS pings (interpolation), ETA countdown on customer tracking screen, "Delivered" confirmation flow with agent signature/photo upload, Push notification when agent is nearby, Edge case handling (agent offline, GPS lost)	Production-ready live tracking

PHASE 4 Admin Dashboard (Weeks 8–9)

Goal: You can manage all orders, menu, agents, coupons, and see analytics from a web dashboard.

Week	Tasks	Deliverable
Week 8	React + Vite + Shadcn project setup, Login screen (admin only), Dashboard home: orders today, revenue, top items, Orders page: list with filters (status, date), Order detail view, Assign delivery agent to order, Menu management: categories + items CRUD with image upload	Admin can manage orders and menu
Week 9	Revenue + orders analytics charts (Recharts), Coupon management (create, list, deactivate), Delivery agents management (list, activate/deactivate), User management (list customers, deactivate), Real-time order bell notification in browser, Deploy to Vercel	Fully operational admin dashboard

PHASE 5 Game Module (Weeks 10–11)

Goal: Game tab live in customer app. Admin can control level rewards.

Week	Tasks	Deliverable
Week 10	game_levels_config, customer_game_profile, game_sessions tables + all game APIs, Level calculation service, Coupon auto-generation with expiry logic, Spin Wheel game screen (animated with React Native Reanimated), Scratch Card screen (finger swipe gesture)	Spin + Scratch games working with coupon generation
Week 11	Food Trivia game screen (3 MCQ per session), Game profile screen (level badge, progress bar, active coupon countdown), Admin dashboard: game level config editor (change discount, expiry per level), Game analytics page (plays/day, coupon usage rate), Push notification when coupon is about to expire	Full game module live + admin configurable

PHASE 6 Polish, Testing & Launch (Week 12)

Area	Tasks
Performance	API response time audit, image compression (WebP), pagination on all list endpoints
Security	Penetration test auth endpoints, rate limit OTP (max 5/hour), validate all inputs server-side
Testing	Happy path E2E test: register → order → pay → track → delivered → review → play game → use coupon
App Store	Expo EAS Build for iOS + Android, TestFlight / Play Console internal testing
Monitoring	Sentry for crash reporting, UptimeRobot for API uptime, Railway logs dashboard
Launch	Soft launch to 10 friends/family, collect feedback, fix critical issues, public launch

Timeline Summary

Phase	Focus	Weeks	Cumulative
1	Foundation + Backend + Auth	1–2	Week 2
2	Customer App — Order Flow	3–5	Week 5
3	Live Tracking + Agent App	6–7	Week 7
4	Admin Dashboard	8–9	Week 9
5	Game Module	10–11	Week 11
6	Polish + Testing + Launch	12	Week 12

i Phase 2 (Customer App) and Phase 4 (Admin Dashboard) can partially overlap if you have a teammate. Phase 5 (Game) is fully independent and can be deferred to post-launch if needed.

8. Final Tech Stack

Layer	Technology	Reason
Customer App	React Native (Expo)	You know React — 80% same syntax, fast to build
Admin Dashboard	React + Vite + Shadcn/ui	Pure React, your comfort zone, instant Vercel deploy
Backend	Spring Boot (Java)	You already know it — no learning curve
Database	PostgreSQL on Railway	Reliable, managed, free tier available
Auth	JWT + BCrypt (cost 12) + Salt	Industry standard, secure, works with Spring Security
Payments	Razorpay	Best for India — UPI, cards, COD support
Real-time	Spring WebSocket (STOMP)	Built into Spring, no extra service needed
Push Notifications	Firebase FCM	Free, reliable, works on Android + iOS
Maps	Google Maps Platform	Best accuracy for India, React Native SDK available
File Storage	AWS S3 or Cloudflare R2	Menu images, food photos, agent uploads
AI Coding	Cursor + Claude	Write component descriptions → get working code
Hosting (API)	Railway	Git-push deploy, free tier, auto SSL
Hosting (Admin)	Vercel	Zero config React deploy
CI/CD	GitHub Actions	Auto build + deploy on push to main

— End of Document —